Optimisation in Deep Learning

Presenter:
Harish G Ramaswamy
IIT Madras

Gradient Descent

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

$$\mathcal{L}(\mathbf{w} + \Delta\mathbf{w}) \approx \mathcal{L}(\mathbf{w}) + \nabla\mathcal{L}(\mathbf{w}) \cdot \Delta\mathbf{w}$$

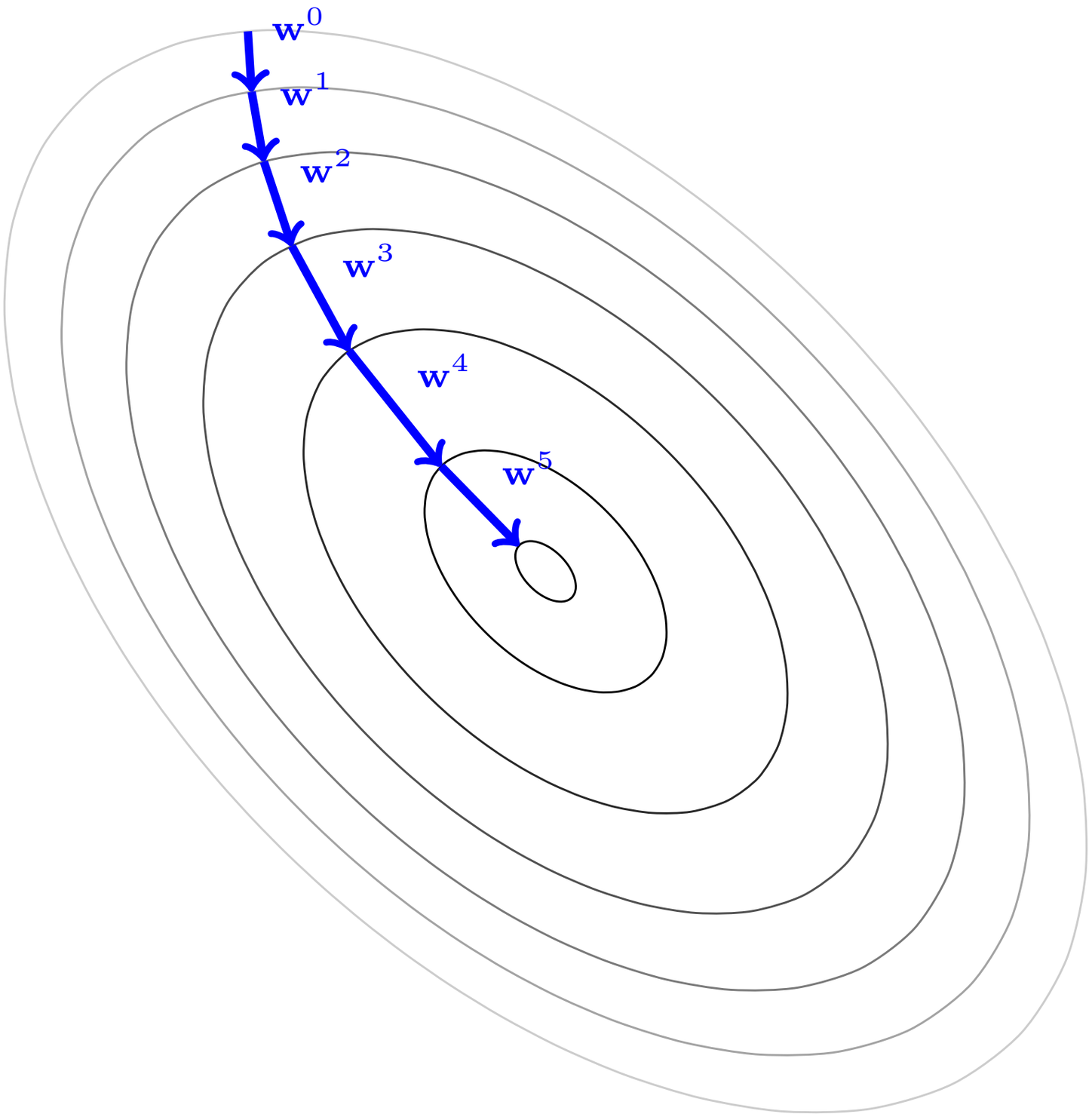
Let $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta\nabla\mathcal{L}(\mathbf{w}^t)$

Then, $\mathcal{L}(\mathbf{w}^{t+1}) \approx \mathcal{L}(\mathbf{w}^t) - \eta\|\nabla\mathcal{L}(\mathbf{w}^t)\|^2$

η is small $\implies$ approximation is valid

η is large and approximation is valid $\implies$ large reduction in loss

Gradient Descent Trajectories



AdaGrad: Dynamic Vector Learning Rate

$$\text{Let } \mathcal{L}(\mathbf{w}) = f(w_1) + g(w_2)$$

$$f(w_1) = 100(w_1 - 1)^2$$

$$g(w_2) = \begin{cases} (w_2 - 1)^2 & \text{w.p. } \frac{1}{10} \\ 0 & \text{w.p. } \frac{9}{10} \end{cases}$$

Each iteration gradient is computed with a sample of f and g.

A common learning rate for both w_1 and w_2 slows convergence.

AdaGrad: Dynamic Vector Learning Rate

Key idea: If accumulated squared partial derivative of a co-ordinate is large, it has moved a lot and likely already near its final solution. So damp these.

AdaGrad

Initialize:

$\mathbf{w}^0$ initial parameters

$\mathbf{s}^0 \leftarrow 0$

$\eta > 0,\ \varepsilon > 0$

Repeat for $t = 1, 2, \ldots, T$:

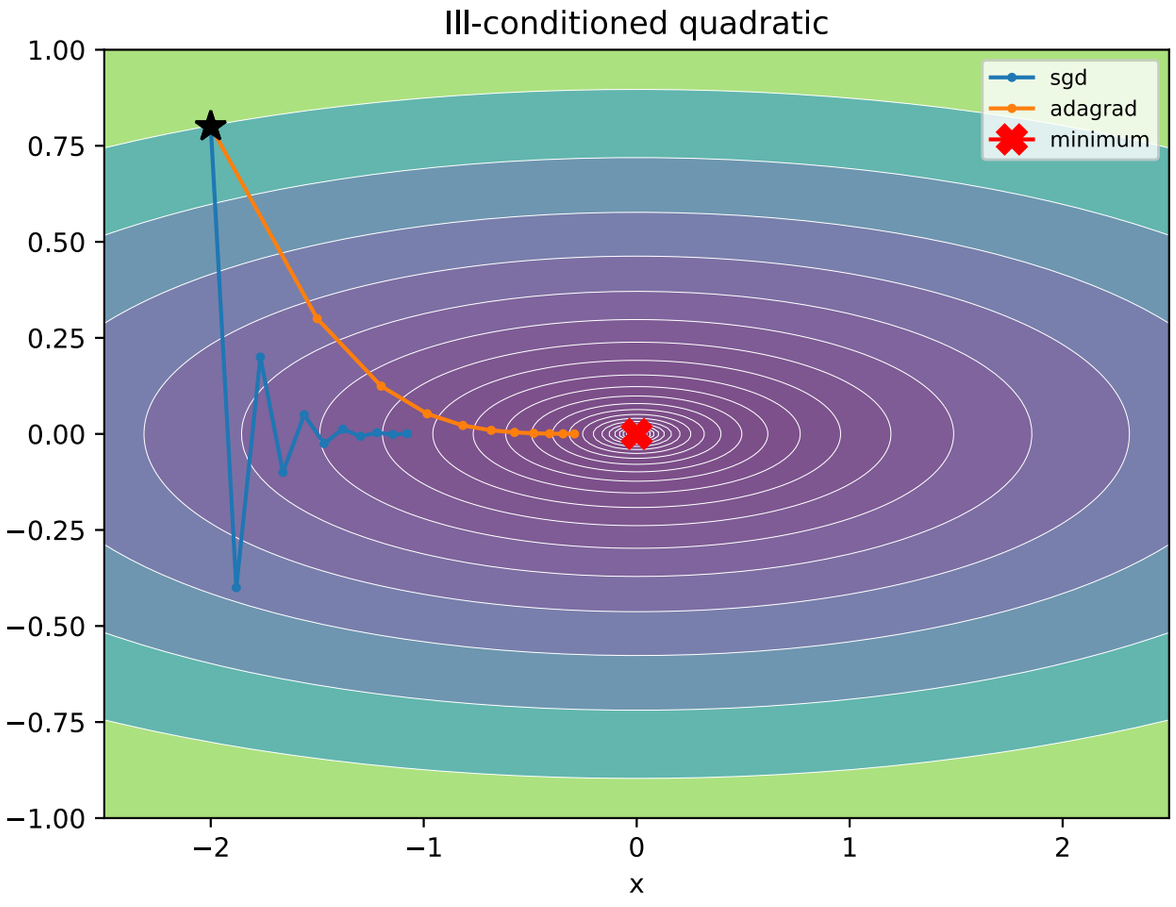
Compute gradient: $\mathbf{g}^t \leftarrow \nabla \mathcal{L}(\mathbf{w}^{t-1})$

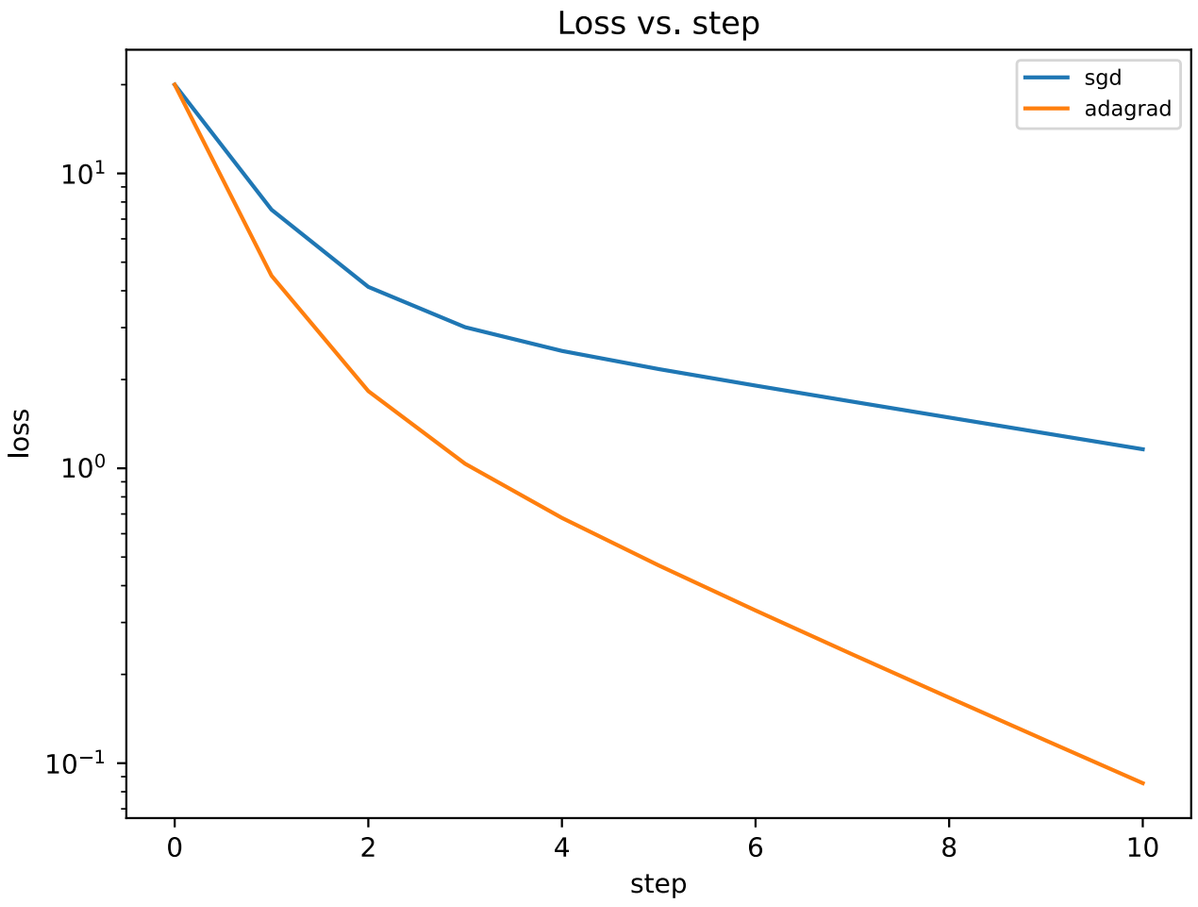
Accumulate squared gradients: $\mathbf{s}^t \leftarrow \mathbf{s}^{t-1} + \mathbf{g}^t \odot \mathbf{g}^t$

Update:

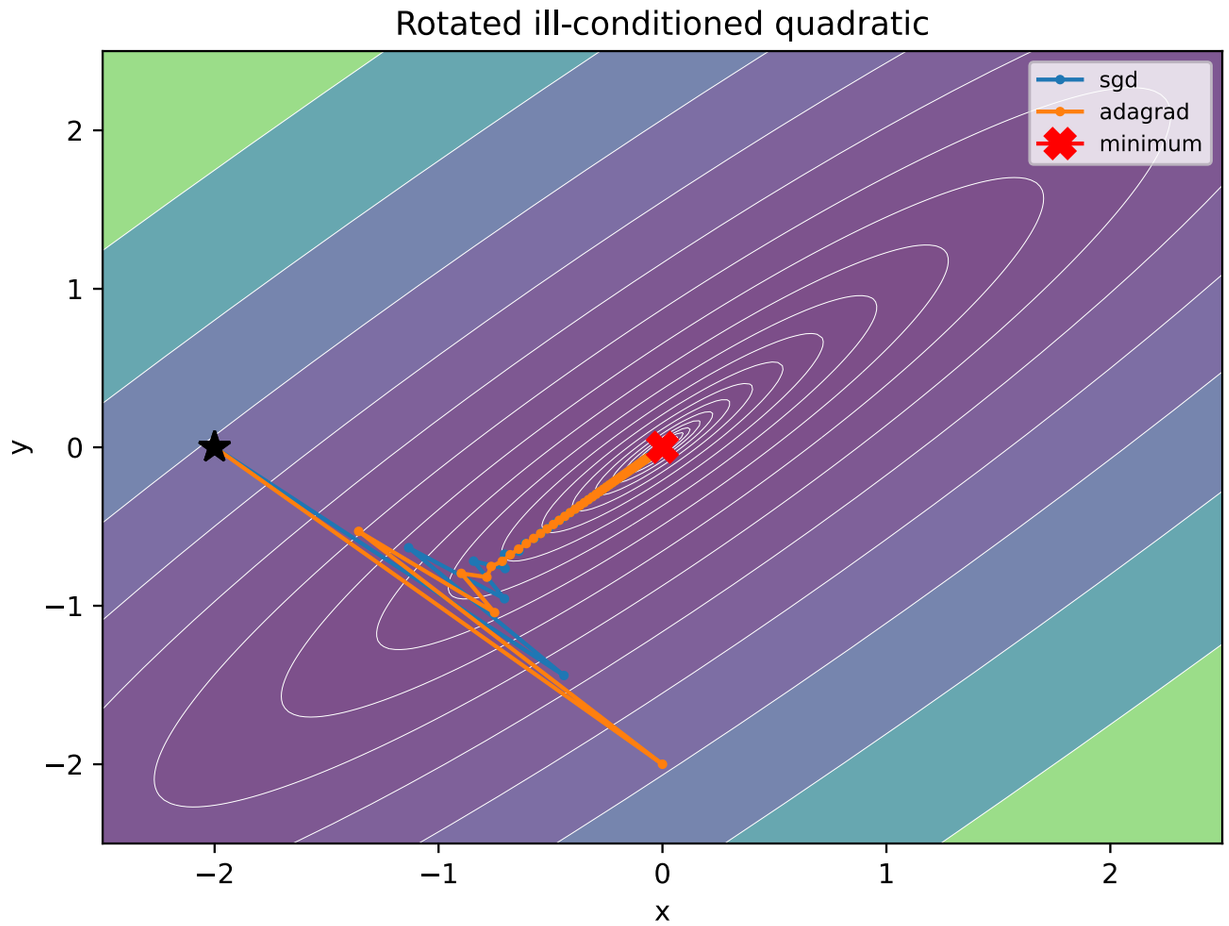
$$\mathbf{w}^t \leftarrow \mathbf{w}^{t-1} - \eta \frac{\mathbf{g}^t}{\sqrt{\mathbf{s}^t} + \varepsilon}$$

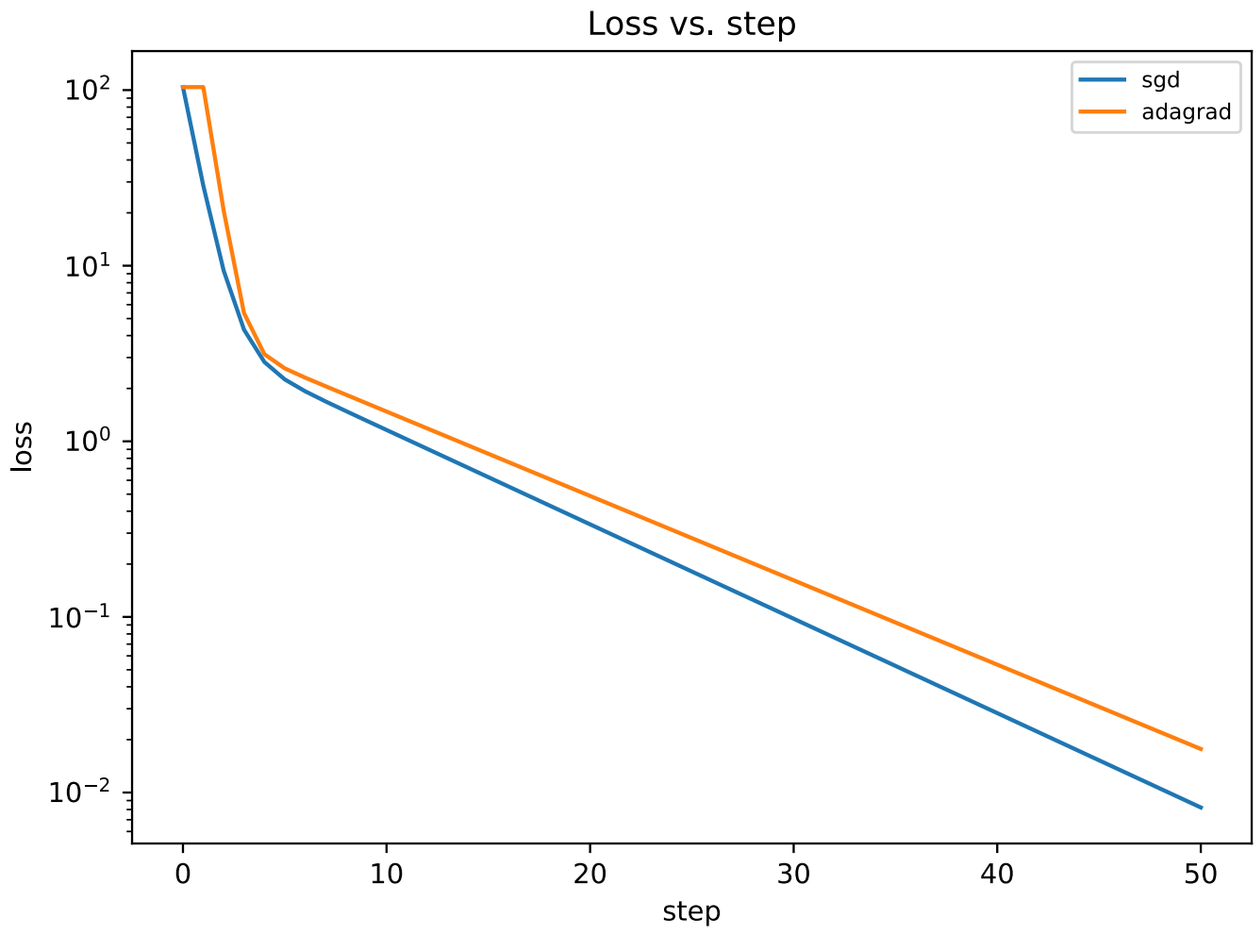
AdaGrad Trajectories





AdaGrad Trajectories





Momentum : Gradient Accumulation

- Let’s step back and take motivation from physics.
- A ball roll down a hill quite fast.
- It is primarily going down due to gravity.
- But the movement is also heavily influenced by momentum.
- Simulated by adding a “momentum” term to gradient update.

$$\mathbf{w}^{t+1} = \mathbf{w}^{t} - \eta \nabla \mathcal{L}(\mathbf{w}^{t}) + \mu(\mathbf{w}^{t} - \mathbf{w}^{t-1})$$

Momentum Pseudocode

Gradient Descent with Momentum

Initialize:

$\mathbf{w}^0$ initial parameters

$\mathbf{v}^0 \leftarrow 0$

$\eta > 0,\ \beta \in (0, 1)$

Repeat for $t = 1, 2, \ldots, T$:

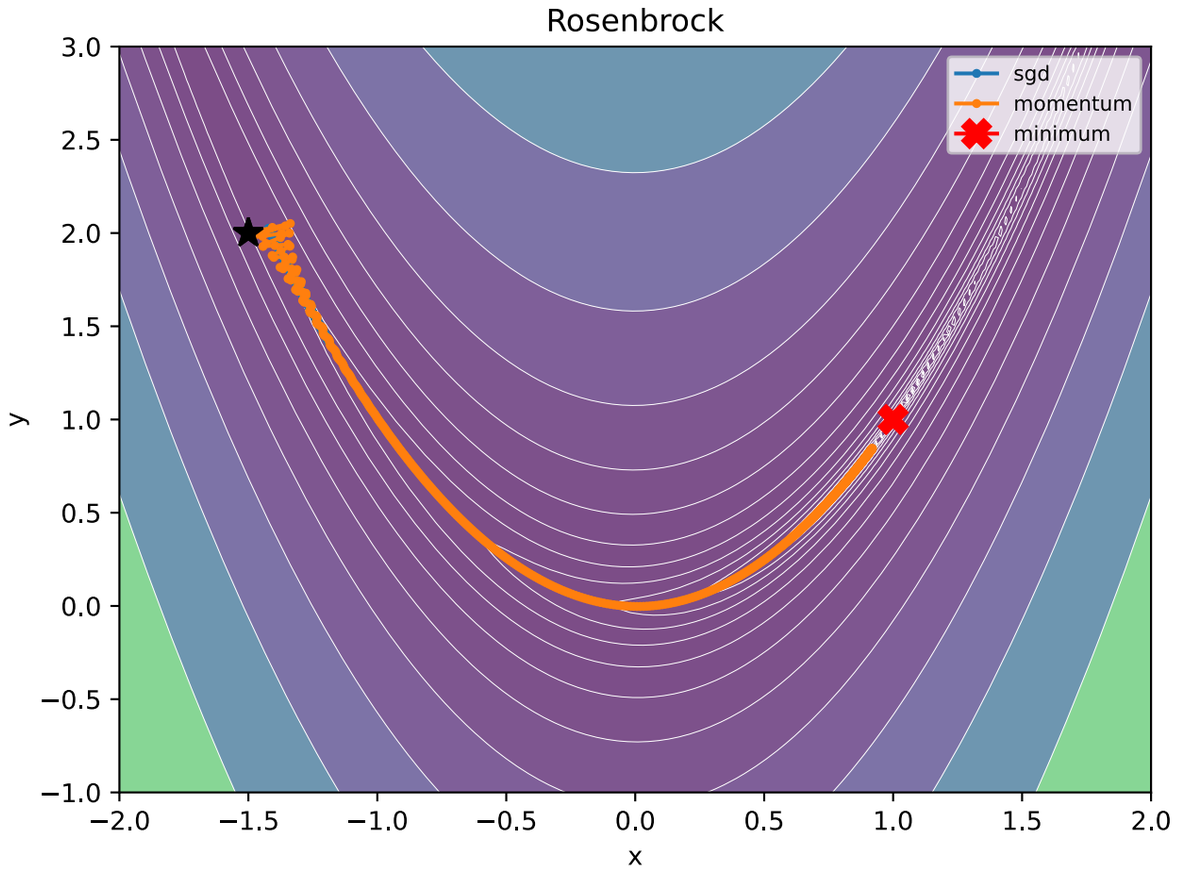
Compute gradient: $\mathbf{g}^t \leftarrow \nabla \mathcal{L}(\mathbf{w}^{t-1})$

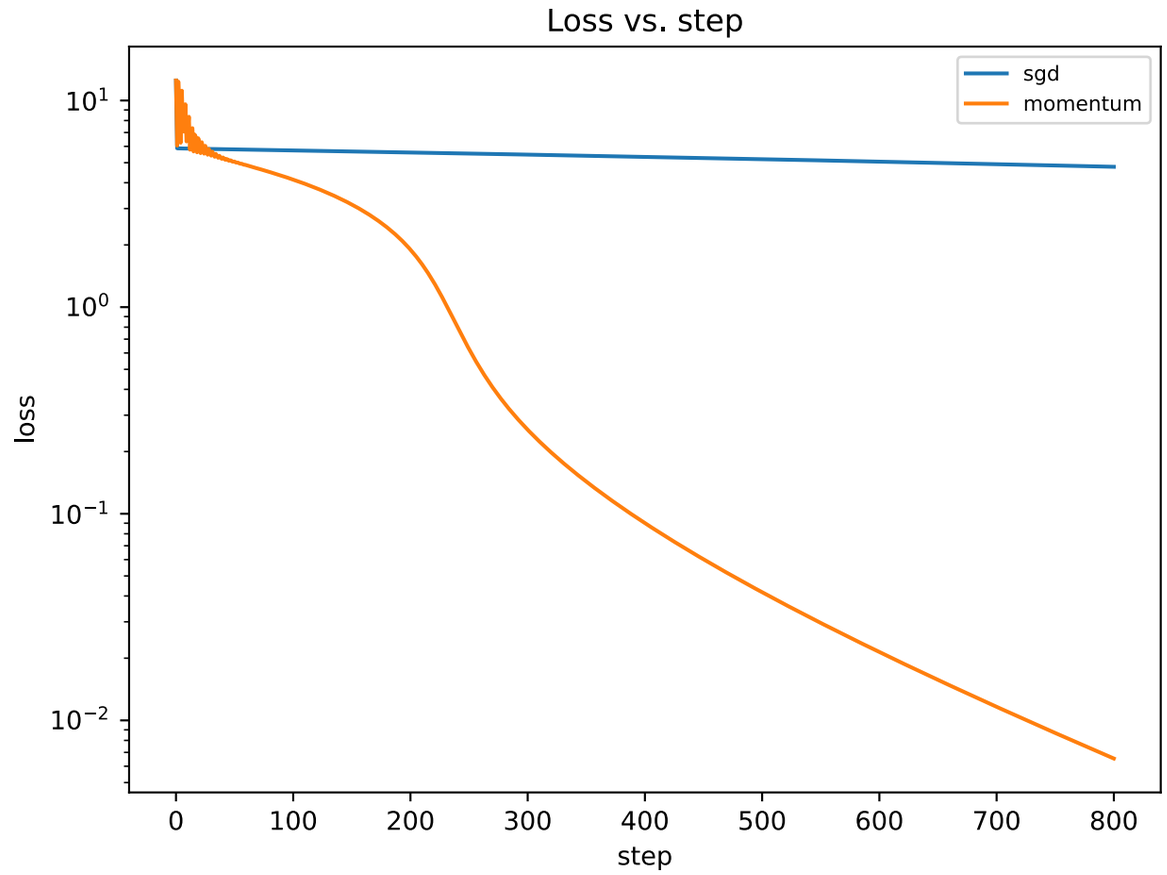
Update velocity: $\mathbf{v}^t \leftarrow \beta \mathbf{v}^{t-1} - \eta \mathbf{g}^t$

Update parameters:

$$\mathbf{w}^t \leftarrow \mathbf{w}^{t-1} + \mathbf{v}^t$$

Momentum Trajectories





Adam : AdaGrad with Momentum

Adam

Initialize:

$\mathbf{w}^0$ initial parameters

$\mathbf{m}^0 \leftarrow 0, \quad \mathbf{v}^0 \leftarrow 0$

$\eta > 0,\ \beta_1, \beta_2 \in (0,1),\ \varepsilon > 0$

Repeat for $t = 1, 2, \ldots, T$:

Compute gradient: $\quad \mathbf{g}^t \leftarrow \nabla f(\mathbf{w}^{t-1})$

First moment: $\quad \mathbf{m}^t \leftarrow \beta_1 \mathbf{m}^{t-1} + (1-\beta_1)\mathbf{g}^t$

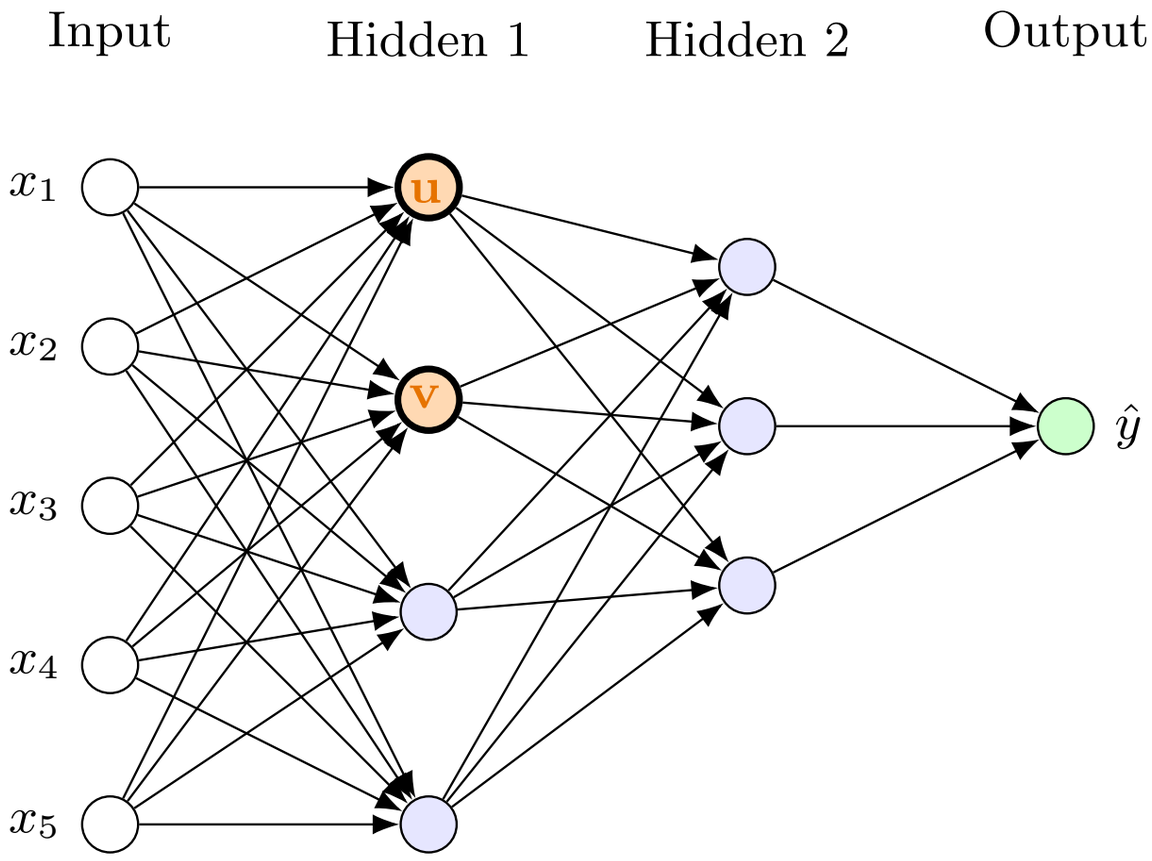
Second moment: $\quad \mathbf{v}^t \leftarrow \beta_2 \mathbf{v}^{t-1} + (1-\beta_2)(\mathbf{g}^t \odot \mathbf{g}^t)$

Bias correction: $\quad \hat{\mathbf{m}}^t \leftarrow \dfrac{\mathbf{m}^t}{1-(\beta_1)^t},\ \hat{\mathbf{v}}^t \leftarrow \dfrac{\mathbf{v}^t}{1-(\beta_2)^t}$

Update:

$$\mathbf{w}^t \leftarrow \mathbf{w}^{t-1} - \eta \frac{\hat{\mathbf{m}}^t}{\sqrt{\hat{\mathbf{v}}^t} + \varepsilon}$$

MuON : Matrix Parameters



Neurons 1,2 have incoming weights u and v.

Often, they both have similar gradients.

In general, matrix parameters tend to have low rank/diversity.

MuON : Matrix Parameters

Key idea: Orthogonalise the matrix update M.

$$\text{Ortho}(M) = \text{argmin}_Q\{\|Q - M\| : QQ^\top = I\}$$

Muon

Input: matrix parameter W^0, stepsize η, momentum $\beta \in (0, 1)$, number of Newton–Schulz steps K

Initialize momentum buffer: $M^0 \leftarrow 0$

for $t = 1, 2, \ldots, T$ **do**

$G^t \leftarrow \nabla f(W^{t-1})$

$M^t \leftarrow \beta M^{t-1} + (1 - \beta)\, G^t$

$Q^t \leftarrow \text{NewtonSchulz}(M^t,\ K)$

$W^t \leftarrow W^{t-1} - \eta\, Q^t$

end for

Nearest Orthogonal Matrix

If $M = USV^\top$ then Ortho$(M) = UV^\top$

SVD works, but too costly.

Newton–Schulz Orthogonalization

Input: matrix M with less rows than columns, Iterations K

Normalize:

$$X_0 \leftarrow \frac{M}{\|M\|_F + \varepsilon}$$

$(a, b, c) = (3.4, -4.7, 2.0)$

for $k = 0, 1, \ldots, K-1$ **do**

$A \leftarrow X_k (X_k)^\top$

$B \leftarrow bA + cAA$

$X_{k+1} \leftarrow aX_k + BX_k$

end for

return X_K

MuON : Newton-Schulz Iteration

$$
\begin{aligned}
X_0 &= USV^\top \\
X_1 &= aX_0 + b(X_0X_0^\top)X_0 + c(X_0X_0^\top)^2X_0 \\
&= U(aS + bS^3 + cS^5)V^\top \\
X_K &= U(\varphi(\varphi(\ldots\varphi(S))))V^\top
\end{aligned}
$$

where $\phi(\sigma) = a\sigma + b\sigma^3 + c\sigma^5$

Note that $V^\top V = I$ and values in S are in $[0, 1]$.

MuON : Newton-Schulz Iteration

1 $\phi(x) = 2x - 1.5x^3 + 0.5x^5$

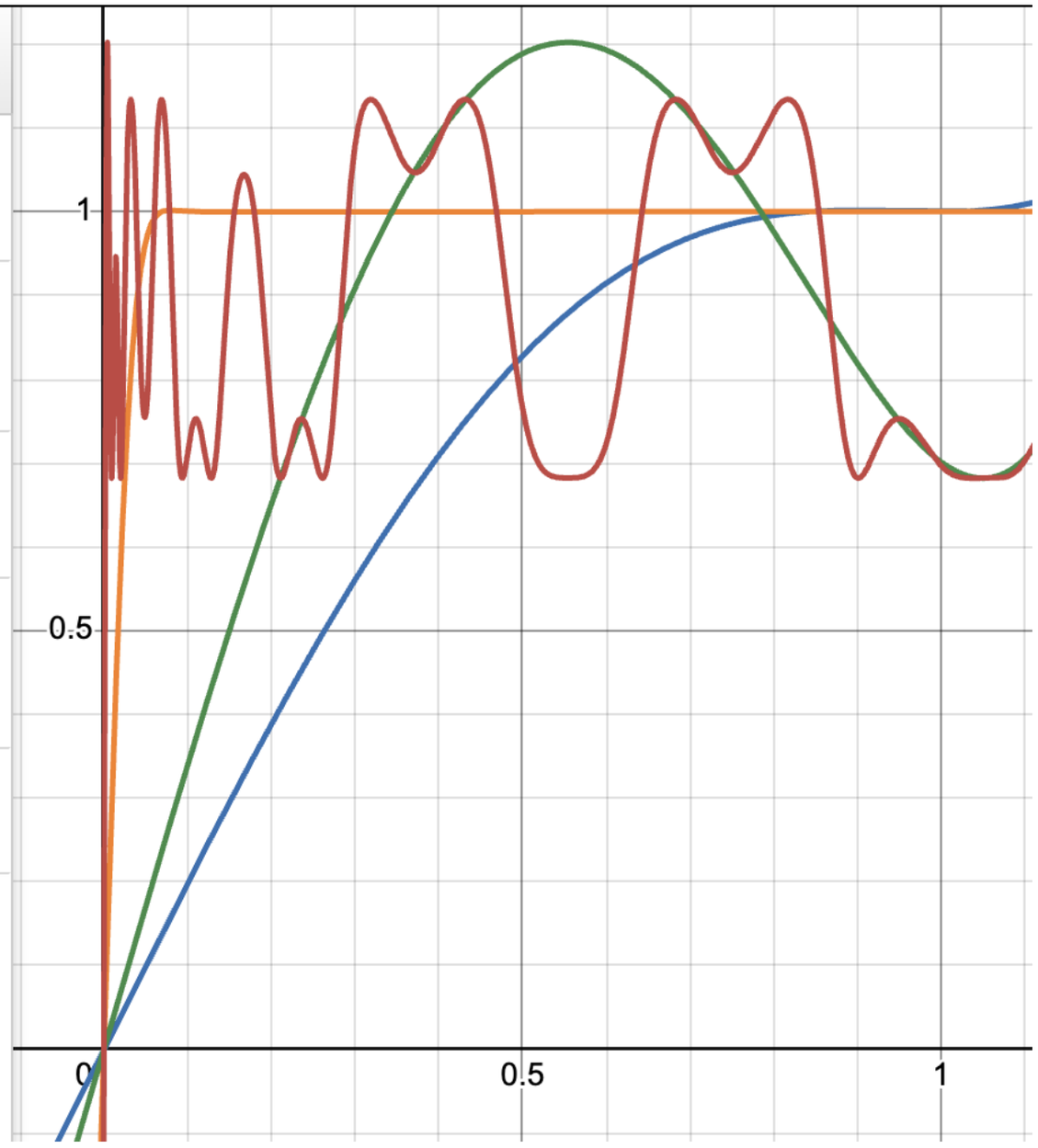
2 $\phi\Big(\phi\Big(\phi\Big(\phi\Big(\phi(x)\Big)\Big)\Big)\Big)$

3 $\rho(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$

4 $\rho\Big(\rho\Big(\rho\Big(\rho\Big(\rho(x)\Big)\Big)\Big)\Big)$

5

6



Jordan et al. 2024

Speedrunning Deep Learning